TP3 — Sécurisation d'un serveur Linux

Mission 1 — Pare-feu firewalld segmenté par zone

1.1 Remise en service

Je réactive firewalld (arrêté au TP1) avec unmask puis enable --now, et je relève la zone par défaut, les zones actives et la zone de chaque connexion NetworkManager.

1.2 Une zone par interface

Une interface pilotée par NetworkManager prend la zone de son profil de connexion : je la règle donc avec nmcli con mod ... connection.zone, puis je réactive chaque connexion. La zone par défaut passe à drop.

- enp0s3 (NAT / Internet) : zone drop, rien n'entre.
- enp0s8 (administration, connexion admin) : zone work, SSH uniquement avec débit limité.
- bond0 (réseau métier) : zone internal, SSH et supervision.
- bond0.42 (VLAN transport) : zone internal, même politique que bond0.

1.3 Ouvrir le strict nécessaire

Dans work, je retire dhcpv6-client et cockpit, j'autorise ssh et je rattache la source 192.168.56.0/24. Dans internal, je retire mdns, samba-client, cockpit et dhcpv6-client, j'autorise ssh et le port 10051/tcp (serveur Zabbix du TP4). Un --reload applique le tout.

1.4 Règles riches, ipset et journalisation

- Sur work, le service ssh est remplacé par une règle riche qui journalise les nouvelles connexions (préfixe SSH-NEW) et les limite à 3/min, avec une rafale de 6/min.
- Le port 10051 est explicitement rejeté sur work : la supervision n'est ouverte que sur le réseau métier.
- Je crée l'ipset helios-blacklist (hash:ip) et une règle drop sur cette liste dans work et dans internal. La liste est alimentée automatiquement en Mission 3.
- --set-log-denied=unicast journalise ce que le pare-feu jette. Un --check-config puis un --reload valident l'ensemble.

1.5 Vérification

Sur srv-core, nft list tables, nft list chain inet firewalld filter_IN_work et nft list set inet firewalld helios-blacklist montrent ce que firewalld a réellement écrit dans nftables (iptables -S n'est qu'une couche de compatibilité). Depuis srv-edge, je teste avec nmap -Pn -p 22,80,10051, nc -zv sur 22 et 10051, puis une rafale de 12 connexions sur le port 22 qui dépasse la limite de 6/min. Les traces SSH-NEW et REJECT|DROP se lisent ensuite dans journalctl -k.

Mission 2 — Durcissement de sshd

2.1 Préparer l'accès par clé

Je crée le groupe système ssh-admins et j'y ajoute noah et julien. Sur srv-edge, je génère une clé ed25519 et je la copie avec ssh-copy-id : la connexion par clé doit fonctionner avant de couper le mot de passe. Je relève ensuite les empreintes des clés d'hôte et je contrôle que authorized_keys est en 0600.

2.2 Fichier de surcharge

Sur Rocky 9, sshd_config commence par un Include de sshd_config.d/*.conf : un fichier de ce dossier l'emporte sur le reste. Le fichier 10-helios-hardening.conf :

- change le port en 2242 et limite l'écoute à 192.168.56.10 et 10.42.0.10 ;
- interdit root et le mot de passe, n'accepte que publickey, avec MaxAuthTries 3 et LoginGraceTime 30 ;
- restreint l'accès au groupe ssh-admins (AllowGroups) ;
- coupe X11, agents et redirections (AllowTcpForwarding no), avec une exception maîtrisée pour le groupe helios-ops : tunnel autorisé uniquement vers localhost:10051 ;
- règle les délais d'inactivité, passe LogLevel à VERBOSE et affiche la bannière /etc/issue.net.

sshd -t valide la syntaxe avant tout redémarrage.

2.3 Algorithmes et crypto-policies

Sur RHEL / Rocky 9, les algorithmes de sshd viennent de la politique cryptographique système : 50-redhat.conf inclut /etc/crypto-policies/back-ends/opensshserver.config. Comme sshd retient la première valeur lue, une directive Ciphers, KexAlgorithms ou MACs n'a d'effet que si elle est lue avant 50-redhat.conf et hors de tout bloc Match. J'ai donc deux approches :

- (a) approche recommandée : durcir la politique système avec update-crypto-policies --set DEFAULT:NO-SHA1 ;
- (b) approche locale : un fichier 05-helios-crypto.conf, dont le nom est inférieur à 50-redhat.conf, qui impose ed25519, curve25519, chacha20-poly1305, aes256-gcm et des MAC sha2-etm.

2.4 Changement de port

Le nouveau port doit être autorisé côté SELinux et côté pare-feu. Je fais donc semanage port -a -t ssh_port_t -p tcp 2242, puis, dans work, je remplace la règle riche du service ssh par la même règle sur le port 2242, et dans internal, j'ouvre 2242/tcp et je retire le service ssh. Je recharge firewalld, puis je redémarre sshd. J'ai fait cette étape avant les redémarrages de la 2.3 : avec Port 2242 actif, sshd ne démarre pas tant que SELinux ne connaît pas le port.

2.5 Les sept tests depuis srv-edge

- 1. Ancien port 22 : refus attendu.
- 2. ssh -p 2242 noah@... par clé : accès réussi.
- 3. Connexion root : refus attendu.
- 4. Connexion par mot de passe : refus attendu.
- 5. Utilisateur nadia, hors ssh-admins : refus attendu.
- 6. Tunnel vers le port 10051 : le refus n'apparaît qu'à l'usage du tunnel.
- 7. Échange de clés diffie-hellman-group14-sha1 : refus attendu (SHA-1 retiré).

Côté srv-core, chaque refus laisse une trace dans journalctl -u sshd, et sshd -T confirme les valeurs retenues (permitrootlogin, passwordauthentication, allowgroups, maxauthtries, port, loglevel).

Mission 3 — Surveillance des journaux et détection

3.1 journald persistant

Le fichier 50-helios.conf de journald.conf.d passe Storage à persistent, active la compression et fixe les limites : 500 Mo au maximum, 1 Go d'espace libre conservé, fichiers de 50 Mo, rétention d'un mois, rotation quotidienne. Après systemctl restart systemd-journald et journalctl --flush, /var/log/journal/<machine-id> existe. Un redémarrage puis journalctl --list-boots montrent que les journaux survivent (au moins deux démarrages).

3.2 Requêtes d'investigation

- journalctl -u sshd.service --since '-2h' -p warning : les avertissements récents de sshd (échecs, refus).

- journalctl _SYSTEMD_UNIT=sshd.service -g 'Invalid user|Failed|Accepted' : les connexions réussies et échouées, avec l'utilisateur et la source.
- journalctl --facility=authpriv --since today : tous les événements d'authentification (sshd, sudo, su).
- journalctl -t sudo -o json-pretty : les élévations de privilèges, avec leurs champs structurés (utilisateur, commande, terminal).
- lastb -n 15 ; last -F -n 10 : les échecs de connexion et les connexions réussies, avec dates complètes.

3.3 auditd

J'active auditd et je charge 90-helios.rules avec augenrules --load. Les règles surveillent les fichiers d'identité (passwd, shadow, group, gshadow), sudoers et sudoers.d, la configuration sshd, le binaire setgid helios-lastlog du TP2, /etc/firewalld/, et toute commande lancée avec les droits root par un compte humain (clé privesc). Je provoque ensuite des événements (création d'un fichier dans sudoers.d, usage de helios-lastlog) et je les exploite avec ausearch -k ... -i et aureport.

3.4 Détection automatique et blocage

Le script /usr/local/sbin/helios-watch-auth.sh, déclenché par helios-watch-auth.timer toutes les 2 minutes (OnCalendar=*:0/2, Persistent=no, AccuracySec=10s), compte les échecs d'authentification SSH par adresse source sur les 10 dernières minutes. Il ajoute à l'ipset helios-blacklist toute adresse qui dépasse 5 échecs. Ses garanties :

- mode strict (set -Eeuo pipefail) et verrou flock : code retour 0 en nominal, 2 si une instance tourne déjà ;
- journalisation dans journald avec le tag helios-watch-auth ;
- liste blanche en dur contenant 192.168.56.1 (poste du formateur) ;
- idempotence : une adresse déjà présente dans /var/lib/helios/blocked.txt ne génère pas de seconde alerte.

Pour tester, srv-edge tente dix connexions avec un utilisateur inexistant (pirate). Sur srv-core, je lance le service, je lis le journal helios-watch-auth et je vérifie que firewall-cmd --ipset=helios-blacklist --get-entries contient 192.168.56.11. Je débloque ensuite srv-edge : timer arrêté, entrée retirée de l'ipset et de blocked.txt, puis timer relancé 11 minutes plus tard avec systemd-run.

Les trois conséquences d'un changement de port SSH sur RHEL 9

- SELinux : le port doit porter l'étiquette ssh_port_t (semanage port -a), sinon sshd ne peut pas l'ouvrir.
- Pare-feu : le service prédéfini ssh ne couvre que le port 22. Il faut ouvrir 2242 dans chaque zone concernée et retirer l'ancien accès.
- Clients et outils : tout ce qui visait le port 22 (clients, scripts, supervision) doit être adapté (ssh -p 2242, scp -P 2242).